

Azure AI Search Scoring Comparison Tool - Reference Document

Overview

This document serves as a comprehensive guide to the Azure AI Search Scoring Comparison Tool. The tool is designed to compare different scoring techniques in Azure AI Search and analyze their impact on search relevance and potential business outcomes.

Table of Contents

1. [Setup Requirements](#setup-requirements)
2. [Script Functionality](#script-functionality)
3. [Scoring Methods Implemented](#scoring-methods-implemented)
4. [Business Impact Analysis](#business-impact-analysis)
5. [Token Usage Tracking](#token-usage-tracking)
6. [Enhancement Opportunities](#enhancement-opportunities)
7. [Running the Script](#running-the-script)

Setup Requirements

Environment Variables

The script requires the following environment variables to be configured in a .env file in the project directory:

AZURE_SEARCH_SERVICE_ENDPOINT="your-search-service-url"

AZURE_SEARCH_INDEX_NAME="your-index-name"

AZURE_SEARCH_API_KEY="your-search-api-key"

Required Python Libraries

The script depends on the following Python libraries:

- azure-search-documents - For Azure AI Search operations
- python-dotenv - For loading environment variables
- tiktoken (optional) - For token usage estimation
- requests - For HTTP requests
- Standard libraries: json, time, os, re, concurrent.futures

You can install these dependencies using pip:

```
pip install azure-search-documents python-dotenv tiktoken requests
```

Script Functionality

Core Features

1. ****Multiple Scoring Methods****: Compares five different scoring approaches for Azure AI Search queries
2. ****Business Impact Analysis****: Quantifies potential business value of different scoring methods
3. ****Token Usage Tracking****: Monitors and reports token consumption for each method
4. ****Performance Measurement****: Tracks and compares query execution times
5. ****Results Summarization****: Generates comprehensive analysis report in a text file

Scoring Methods Implemented

1. Default Azure AI Search Scoring

Uses Azure's built-in BM25 scoring algorithm without any customization.

2. Custom Post-Query Scoring

Applies additional scoring logic to the search results after they are returned from Azure AI Search:

- Boosts results based on custom criteria
- Applies content relevance weighting
- Sets minimum score thresholds

3. Keyword Boosting

Enhances scores for results that contain specific keywords extracted from the search query.

4. Azure Best Practices Scoring Profile

Creates and applies a scoring profile following Azure AI Search best practices:

- Weights key fields appropriately
- Applies freshness scoring (if applicable)
- Implements magnitude scoring for numeric fields (if applicable)

5. Adaptive Query Profile

Creates a dynamic scoring profile that adapts to the search query:

- Extracts keywords from the search query
- Creates a temporary scoring profile that boosts those keywords
- Applies customized field weights

Business Impact Analysis

The script calculates potential business impact using configurable metrics:

- Click probability on top results
- Conversion rates for relevant vs. irrelevant results

- Average transaction value
- Daily search volume

Results are presented as estimated additional revenue and relevance improvement percentages.

Token Usage Tracking

The script includes a TokenTracker class that estimates token usage for each search method:

- Tracks query tokens
- Tracks options/parameter tokens
- Tracks response tokens
- Calculates total token consumption

This helps optimize for cost-efficiency when using consumption-based Azure OpenAI services.

Running the Script

1. ****Setup Environment****:

- Configure the .env file with your Azure AI Search credentials
- Install required dependencies

2. ****Execute the Script****:

- Run the Jupyter notebook search.ipynb
- Or execute the code as a Python script

3. ****Enter Search Query****:

- The script will prompt for a search query
- A default query will be used if none is provided

4. ****Review Results****:

- The script will display detailed results for each scoring method
- A comprehensive analysis will be written to results_summary.txt
- Business impact metrics and token usage will be summarized

Example Command to Execute

If converted to a Python script:

```
python search.py
```

Or within Jupyter Notebook:

```
jupyter notebook search.ipynb
```

Code Modification Guidelines

When modifying the code to enhance search results:

1. ****Focus on Scoring Profiles****:

- The `azure_search_best_practices` and `adaptive_query_profile` methods show the most promise for improvement
- Adjust field weights in these profiles based on your specific content

2. ****Tune Business Metrics****:

- Customize the `business_metrics` dictionary with values specific to your business
- This will provide more accurate revenue impact estimates

3. ****Custom Post-Query Scoring****:

- The `apply_custom_scoring` function can be extended with domain-specific logic
- Consider adding semantic similarity measures for more intelligent ranking

4. ****Keyword Extraction****:

- Enhance the keyword extraction logic to be more sophisticated than simple word splitting
- Consider using NLP techniques to identify key concepts

5. ****Add Facet Analysis****:

- Implement facet-based scoring adjustments for better categorized results
- Compare the impact of facet-aware scoring

By implementing these enhancements, you can significantly improve search relevance and business outcomes beyond what the current script demonstrates.